

Full-Stack AI Engineer Assignment

Project: Policy Pilot

1. Assignment Overview

Target level: Senior Full-Stack AI Engineer

Expected build time: Approximately one week

Submission deadline: 24 hours before the technical review

Submission format: Git repository plus supporting documentation in Markdown or PDF

You may use standard open-source frameworks, public documentation, diagramming tools, and AI coding assistants. Please disclose where AI-assisted development was used.

The technical review will be approximately 60 minutes:

- 15-minute implementation walkthrough
 - 30 to 45 minutes of system-design discussion, technical defense, and Q&A
-

2. Business Scenario

At a large enterprise, employees regularly submit self-service access requests, such as:

- “Grant me EDIT access to the Finance analytics dataset.”
- “Add me to the production deployer group.”
- “Provide temporary access to a restricted reporting environment.”

Today, an IT administrator must manually:

1. Look up the requester's existing entitlements in a relational registry.
2. Review long, unstructured data-governance and least-privilege policy documents.
3. Determine whether the request should be approved, denied, or escalated.
4. Document the reasoning for audit and compliance purposes.

This process is slow, inconsistent, difficult to scale, and hard to audit.

Your assignment is to design and build the core execution flow for **Policy Pilot**, a full-stack, AI-assisted access-review system.

Policy Pilot should:

- Receive access-request events.
- Retrieve the requester's current entitlements.
- Use a Retrieval-Augmented Generation, or RAG, pipeline to identify relevant policy requirements.
- Generate a structured recommendation.
- Present that recommendation to a human reviewer.
- Record the reviewer's final decision and rationale.

The system must remain human-in-the-loop. The AI may recommend an outcome, but it must never directly grant or revoke access.

3. Required Technology

Use the following core stack:

Backend

- TypeScript
- Node.js
- NestJS

Frontend

- React
- TanStack React Query

Data and Infrastructure

- PostgreSQL
- A vector database or vector-search implementation
- An asynchronous queue or queue abstraction

AI Layer

- Any hosted large language model
- Embedding model of your choice
- Structured, schema-validated model output

Downstream enterprise systems may be mocked. However, the core workflow must run end-to-end using sample data.

Reasonable technology substitutions are acceptable when clearly explained and defended.

4. Required Execution Flow

Build a working vertical slice that demonstrates the following capabilities.

4.1 Access-Request Ingestion

Create a webhook endpoint that accepts access-request events.

The ingestion flow must:

- Validate the incoming payload.
- Process requests asynchronously.
- Enqueue each request for processing.
- Prevent duplicate processing using the request ID or an idempotency key.
- Return an appropriate response without waiting for the full AI workflow to complete.

4.2 Entitlement Lookup

Query a PostgreSQL entitlement registry to retrieve the requester's current active entitlements.

The lookup should support evaluating:

- Existing access to the target system
- Potential duplicate access
- Separation-of-duties conflicts
- Other contextual information needed by the recommendation workflow

4.3 Policy RAG Pipeline

Implement a RAG pipeline over policy documents in PDF or comparable long-form format.

The pipeline should include:

1. Document parsing
2. Chunking
3. Metadata tagging
4. Embedding generation
5. Vector storage
6. Query-time retrieval
7. Metadata filtering
8. Source citation

Document and defend your chunking strategy.

Relevant metadata may include:

- Document name
- Document version
- Section number
- Policy domain
- Department
- Cost center
- Target system
- Entitlement type
- Effective date

The recommendation must cite the specific policy passages used to support its conclusion.

4.4 Recommendation Agent

Create an AI-powered recommendation service that returns schema-validated JSON.

The response must include:

```
JSON
{
  "decision": "APPROVE | DENY | ESCALATE",
  "justification": "string",
  "policy_citations": [
    {
      "document_name": "string",
      "section": "string",
      "excerpt": "string"
    }
  ],
  "confidence": 0.0
}
```

You may extend the schema with additional fields when useful.

The system must:

- Validate the model response before persisting it.
- Reject or retry malformed responses.
- Keep prompt templates version-controlled.
- Clearly separate retrieved policy context from untrusted request content.

- Avoid presenting unsupported claims as policy requirements.
- Escalate when the available evidence is insufficient or ambiguous.

4.5 Human-in-the-Loop Dashboard

Build a minimal React dashboard for reviewing pending access recommendations.

The dashboard should allow a reviewer to:

- View pending access requests.
- View the requester's current entitlements.
- View the AI recommendation.
- Review the justification and cited policy passages.
- Approve the recommendation.
- Deny the request.
- Override the AI recommendation.
- Provide an override reason or reviewer comment.

Use TanStack React Query for server-state management.

The UI should include appropriate loading, empty, success, and error states.

The human decision must be the only path that could trigger an access change. A mocked execution adapter is sufficient for this assignment.

4.6 Auditability and Idempotency

Design the data model so that:

- The same request cannot be executed more than once.
- AI recommendations are preserved.
- Human decisions are preserved separately from AI recommendations.
- Overrides include the reviewer's rationale.
- Audit records are append-only.
- Important state transitions can be reconstructed.

4.7 Operational Guardrails

Demonstrate or document how the system handles:

- Requests arriving faster than downstream systems can process them
- Retry behavior
- Backoff and jitter
- At-least-once queue delivery
- Duplicate events

- Downstream rate limits
- Partial failures
- LLM timeouts
- Invalid structured output
- Sensitive-data handling

Assume the following operating conditions:

- Downstream systems allow approximately 60 requests per minute.
- Policy Pilot may receive bursts of up to 300 events per minute.
- Employee IDs and cost-center values must be masked in application logs and distributed traces.
- The LLM must never receive credentials or directly execute a mutation.

4.8 Evaluation Framework

Create a small, version-controlled golden dataset containing representative access requests and expected behavior.

The dataset should include cases such as:

- A clearly permitted request
- A clearly prohibited request
- A request requiring secondary approval
- A separation-of-duties conflict
- A request with insufficient policy evidence
- A request where the user already has the requested entitlement
- An invalid or incomplete request
- A prompt-injection attempt contained within user-provided justification

Provide an evaluation script that reports:

- Structured-output schema validity
- Retrieval quality
- Grounding or faithfulness
- Decision correctness
- Citation quality
- Latency
- Estimated model cost

The evaluation should be runnable using a single documented command.

5. Sample Access-Request Payload

JSON

```
{
  "request_id": "req_access_2026_44821",
  "employee_id": "EMP-52190",
  "request_type": "GRANT_ENTITLEMENT",
  "timestamp": "2026-07-01T09:15:00Z",
  "requester": {
    "title": "Data Analyst",
    "department": "Finance Analytics",
    "cost_center": "CC-FIN-07"
  },
  "target": {
    "system_name": "DATA_WAREHOUSE",
    "entitlement_key": "FIN_DATASET_EDIT",
    "justification": "Need to build quarterly revenue models."
  }
}
```

6. Existing Entitlement Registry

Assume the following PostgreSQL table already exists and must be queried by your application.

Column	Type	Description
registry_id	BIGINT	Primary key
employee_id	VARCHAR(50)	Indexed employee identifier
system_name	VARCHAR(50)	Target application or platform
entitlement_key	VARCHAR(100)	Permission or entitlement identifier
granted_date	TIMESTAMP	Date the entitlement was originally granted
is_active	BOOLEAN	Whether the entitlement is currently active

You may create additional tables for requests, recommendations, decisions, idempotency, audit history, document metadata, and evaluation results.

7. Example Policy Content

The policy documents may contain language similar to the following:

Employees in cost center CC-FIN-07 may be granted read access to Finance analytics datasets under standard approval. Write or EDIT access to production Finance datasets requires secondary approval from a Data Governance owner and must be time-boxed to 90 days. Any request that would grant a requester both dataset EDIT and billing-export entitlements simultaneously must be denied as a separation-of-duties conflict.

Your system should identify the relevant portions of this policy, compare them with the request and existing entitlements, and produce an appropriately grounded recommendation.

For the sample request, a well-supported recommendation would likely be **ESCALATE**, because production EDIT access requires secondary approval and a time-boxed entitlement.

8. Required Deliverables

Submit the following materials 24 hours before the technical review.

8.1 Source Code

A runnable repository containing:

- NestJS backend
- React frontend
- PostgreSQL schema and migrations
- Policy ingestion and retrieval implementation
- Recommendation workflow
- Human-review workflow
- Golden dataset
- Evaluation script
- Automated tests
- Docker configuration
- CI configuration

8.2 System Architecture Blueprint

Provide a diagram or clearly structured text flow showing the boundaries between:

- React dashboard
- NestJS API
- Webhook ingestion layer
- Asynchronous queue
- Processing worker
- PostgreSQL
- Vector store
- Embedding endpoint
- LLM endpoint
- Mock downstream execution adapter
- Logging, metrics, and tracing components

The diagram should make the asynchronous flow and human-approval boundary clear.

8.3 Technical Implementation Brief

Provide a technical brief of no more than two pages covering:

- Architecture overview
- PDF parsing strategy
- Chunking strategy and trade-offs
- Metadata-tagging approach
- Retrieval and filtering strategy
- Prompt and structured-output design
- Idempotency design
- Append-only audit strategy
- Downstream rate-limit handling
- Retry and backpressure strategy
- PII-masking approach
- Evaluation and observability strategy
- Major assumptions and trade-offs

8.4 Core Service Example

Highlight one concise NestJS service or processing component that demonstrates either:

- The asynchronous request-processing loop, or
- Structured LLM response validation before persistence

The code should demonstrate production-minded error handling, typing, validation, and clear separation of responsibilities.

8.5 Working Demonstration

Provide instructions for running the system locally.

The reviewer should be able to:

1. Start the required services.
2. Load sample policy documents.
3. Submit an access request.
4. Observe asynchronous processing.
5. View the recommendation in the dashboard.
6. Review the cited policy evidence.
7. Approve, deny, or override the recommendation.
8. View the resulting audit history.

8.6 Evaluation Report

Include results for the golden dataset, including:

- Number of test cases
 - Schema-validation success rate
 - Retrieval results
 - Grounding results
 - Decision accuracy
 - Citation accuracy
 - p50 and p95 latency
 - Estimated cost
 - Known failure modes
-

9. Evaluation Criteria

Full-Stack Architecture and Integration: 25%

We will evaluate:

- Clear separation of frontend, API, worker, data, and AI responsibilities
- End-to-end integration
- Appropriate asynchronous architecture
- Scalable and maintainable component boundaries
- Effective use of React, NestJS, PostgreSQL, and queue-based processing

Applied AI and Automation: 25%

We will evaluate:

- Document parsing and chunking decisions
- Metadata design
- Retrieval quality
- Grounded recommendations
- Structured-output validation
- Prompt containment
- Source citations
- Clear human-in-the-loop separation

Engineering Craftsmanship and Observability: 25%

We will evaluate:

- Type safety
- Input and output validation
- Maintainable code structure
- Error handling
- Automated tests
- Idempotency
- Audit logging
- PII masking
- AI observability and evaluations

Communication and Technical Judgment: 15%

We will evaluate:

- Ability to explain architectural decisions
- Understanding of trade-offs
- Ability to identify risks and limitations
- Clarity of documentation
- Constructive technical reasoning during the review

Production Readiness: 10%

We will evaluate:

- Rate-limit handling
- Retry and backpressure design
- Containerization
- CI pipeline
- Database choices
- Security boundaries

- Deployment and scaling considerations
-

10. Optional Stretch Goals

These items are not required but may strengthen the submission:

- Add an LLM-as-judge grounding evaluation to CI.
 - Implement automatic expiration for time-boxed entitlements.
 - Detect separation-of-duties conflicts using retrieved policy rules.
 - Add role-based access control for reviewers.
 - Add OpenTelemetry traces across the request lifecycle.
 - Add prompt-version comparison or rollback support.
 - Add dead-letter queue handling and replay tooling.
 - Add a reviewer feedback loop that captures disagreement with AI recommendations.
 - Add an administrative view for policy-document versions and ingestion status.
-

Submission Self-Review Checklist

Complete this checklist before submitting your work.

Backend

- Incoming webhook payloads are validated.
- API routes use appropriate authentication and authorization.
- Request ingestion is asynchronous.
- Duplicate webhook events do not create duplicate processing or execution.
- DTOs and external payloads are schema-validated.
- Errors return consistent responses without leaking stack traces or secrets.
- Core services have automated unit tests.
- Retryable and non-retryable failures are handled differently.

Frontend

- TanStack React Query is used for server state.
- React components do not call `fetch` or Axios directly.
- Network access is encapsulated in a configured API client and reusable hooks.
- Protected routes and permission checks are implemented.

- Loading, empty, error, and success states are handled.
- Reviewers can approve, deny, and override recommendations.
- Overrides require a reviewer explanation.

AI and RAG

- Policy answers and recommendations cite their source passages.
- Parsing and chunking strategies are documented.
- Retrieved chunks retain useful source metadata.
- Metadata filters are applied where appropriate.
- Prompt templates are version-controlled.
- Model output is schema-validated before use or persistence.
- Invalid structured output is retried or safely rejected.
- The system escalates when evidence is incomplete or conflicting.
- The LLM cannot execute access changes.
- User-provided content is treated as untrusted input.
- Credentials and sensitive secrets are never included in model context.

Human-in-the-Loop Controls

- AI recommendations are stored separately from human decisions.
- The human reviewer is the final decision-maker.
- A human action is required before any access mutation.
- Override decisions and explanations are recorded.
- The final action can be traced to the responsible reviewer.

Evaluations

- A representative golden dataset is committed to the repository.
- The dataset includes approval, denial, escalation, conflict, and insufficient-evidence cases.
- The evaluation script runs from one documented command.
- Schema validity is reported.
- Retrieval or citation quality is reported.
- Grounding or faithfulness is reported.
- Decision correctness is reported.
- Latency and estimated cost are reported.
- Known failure cases are documented.

Production Readiness

- The application includes a multi-stage Dockerfile.

- Containers run as a non-root user.
- CI runs linting, type-checking, tests, and builds.
- PostgreSQL migrations are included.
- An idempotency mechanism prevents duplicate execution.
- Audit records are append-only.
- Queue consumers are safe under at-least-once delivery.
- Retry behavior includes backoff and jitter.
- Downstream rate limits and backpressure are handled.
- Employee IDs and cost centers are masked in logs and traces.
- LLM latency, token usage, cost, and failures are observable.
- Health and readiness endpoints are included or documented.

Documentation and Submission

- The repository includes clear local setup instructions.
- The architecture blueprint is included.
- The technical implementation brief is no more than two pages.
- Major assumptions and trade-offs are documented.
- Sample data and policy documents are included.
- The full execution flow can be demonstrated end-to-end.
- AI-assisted development is disclosed.
- All deliverables are submitted at least 24 hours before the technical review.